

## PDF相关库

PdfiumAndroid

jni

Mutex.h

mainJNLib.cpp

include下的fpdf文件

Android.mk & Application.mk

AndroidPdfViewer

CacheManager

RenderingHandler

绘制过程

PdfRenderer

Pdfium (编译前的c++库)

具体到项目中实现

# PDF相关库

---

## PdfiumAndroid

---

[Github源地址](#)

初始化编译需要修改gradle参数

### 1. build.gradle

```
buildscript {
    repositories {
        //jcenter已经过期
        //jcenter()
        //google()
        //mavenCentral()
        maven { url 'https://jitpack.io' }
        maven { url 'https://maven.aliyun.com/repository/central' }
        maven { url 'https://maven.aliyun.com/repository/jcenter' }
        maven { url 'https://maven.aliyun.com/repository/public' }
        maven { url 'https://maven.aliyun.com/repository/google' }
    }
    ...
    dependencies {
        //提高gradle编译版本
        classpath 'com.android.tools.build:gradle:4.0.1'
        ...
    }
}

repositories {
    //    jcenter()
    //    google()
```

```
// mavenCentral()
maven { url 'https://jitpack.io' }
maven { url 'https://maven.aliyun.com/repository/central' }
maven { url 'https://maven.aliyun.com/repository/jcenter' }
maven { url 'https://maven.aliyun.com/repository/public' }
maven { url 'https://maven.aliyun.com/repository/google' }
}
```

## 2. gradle/wrapper/gradle-wrapper.properties

```
...
distributionUrl=https\://services.gradle.org/distributions/gradle-6.1.1-bin.zip
...
```

## 3. gradle jdk版本为corretto-1.8

4. 使用ndk-build的时候，还需在Application.mk中去掉mips, mips64

```
APP_STL := c++_shared
APP_CPPFLAGS += -fexceptions

#For ANativewindow support
APP_PLATFORM = android-14

APP_ABI := armeabi-v7a \
           arm64-v8a \
           x86 \
           x86_64

#           mips \
#           mips64 \
```

## jni

### Mutex.h

线程互斥锁，确保同一时间内只有一个线程可以访问

### mainJNILib.cpp

- 需要结合目录下的util.hpp
  - 特别是**JNI\_FUNC**和**JNI\_ARGS**，实际就是define后替换了 JNIEXPORT JNICALL  
Java\_com\_shockwave\_pdfium\_##bindClass####name 以及 JNIEnv \*env, jobject thiz

```
#define JNI_FUNC(retType, bindClass, name) JNIEXPORT retType JNICALL
Java_com_shockwave_pdfium_##bindClass##_##name
#define JNI_ARGS    JNIEnv *env, jobject thiz
```

- nativeLoadPage(JNI\_ARGS, jlong docPtr, int pageIndex)

- 首先通过`reinterpret_cast`关键字对传入的`docPtr`（也即`PdfDocument`的native层指针`PdfDocument.mNativeDocPtr`）进行类型转换，并且初始化出一个`DocumentFile`类，并且对这个类进行线程锁和初始化FPDF库（具体可查`mainJNLib.cpp#DocumentFile`和`initLibraryIfNeed`）
- 接着通过`loadPageInternal`方法将获得的`docPtr`和`pageIndex`转成一个`FPDF_PAGE`类，也就是pdf的第`pageIndex`页的实际像素数据，最后返回这块数据的数组指针。这样的话，在java层中就可以通过`pdfiumCore.renderPageBitmap`获取到这一页的bitmap。

```
// fpdfview.h
// Function: FPDF_LoadPage
//          Load a page inside the document.
// Parameters:
//          document    - Handle to document. Returned by FPDF_LoadDocument
//          page_index  - Index number of the page. 0 for the first page.
// Return value:
//          A handle to the loaded page, or NULL if page load fails.
// Comments:
//          The loaded page can be rendered to devices using FPDF_RenderPage.
//          The loaded page can be closed using FPDF_ClosePage.
DLLEXPORT FPDF_PAGE STDCALL FPDF_LoadPage(FPDF_DOCUMENT document,
                                           int page_index);

// https://github.com/documentcloud/pdfium/blob/master/fpdfsdk/fpdfview.cpp
// fpdfview.cpp
DLLEXPORT FPDF_PAGE STDCALL FPDF_LoadPage(FPDF_DOCUMENT document,
                                           int page_index) {
    UnderlyingDocumentType* pDoc = UnderlyingFromFPDFDocument(document);
    if (!pDoc)
        return nullptr;

    if (page_index < 0 || page_index >= pDoc->GetPageCount())
        return nullptr;

#ifdef PDF_ENABLE_XFA
    return pDoc->GetXFAPage(page_index);
#else // PDF_ENABLE_XFA
    CPDF_Dictionary* pDict = pDoc->GetPage(page_index);
    if (!pDict)
        return nullptr;

    CPDF_Page* pPage = new CPDF_Page(pDoc, pDict, true);
    pPage->ParseContent();
    return pPage;
#endif // PDF_ENABLE_XFA
}
```

- nativeClosePage
- nativeRenderPageBitmap
  - 实际渲染将pdf的某页进行渲染的方法

```
JNI_FUNC(void, PdfiumCore, nativeRenderPageBitmap)(JNI_ARGS, jlong pagePtr, jobject bitmap,
```

```

        jint dpi, jint startX, jint startY,
        jint drawSizeHor, jint drawSizeVer,
        jboolean renderAnnot){

//通过pagePtr指针来获取当前pdf文件的某页的数据
FPDF_PAGE page = reinterpret_cast<FPDF_PAGE>(pagePtr);

...

//初始化检测用于存放pdf某页数据的bitmap是否符合
AndroidBitmapInfo info;
int ret;
if((ret = AndroidBitmap_getInfo(env, bitmap, &info)) < 0) {
    LOGE("Fetching bitmap info failed: %s", strerror(ret * -1));
    return;
}

int canvasHorSize = info.width;
int canvasVerSize = info.height;

//565相对于8888来说，刚好两倍，方便数组对齐
//RGB_565只占16位，没有透明度；RGB_8888占32位，有透明度
if(info.format != ANDROID_BITMAP_FORMAT_RGBA_8888 && info.format !=
ANDROID_BITMAP_FORMAT_RGB_565){
    LOGE("Bitmap format must be RGBA_8888 or RGB_565");
    return;
}

void *addr;
//锁bitmap像素数组
if( (ret = AndroidBitmap_lockPixels(env, bitmap, &addr)) != 0 ){
    LOGE("Locking bitmap failed: %s", strerror(ret * -1));
    return;
}

//根据不同的bitmap的像素格式，进行不同的内存空间申请
void *tmp;
int format;
int sourceStride;
if (info.format == ANDROID_BITMAP_FORMAT_RGB_565) {
    tmp = malloc(canvasVerSize * canvasHorSize * sizeof(rgb));
    sourceStride = canvasHorSize * sizeof(rgb);
    format = FPDFBitmap_BGR;
} else {
    tmp = addr;
    sourceStride = info.stride;
    format = FPDFBitmap_BGRA;
}

FPDF_BITMAP pdfBitmap = FPDFBitmap_CreateEx( canvasHorSize, canvasVerSize,
                                                format, tmp, sourceStride);

/*LOGD("Start X: %d", startX);

```

```

LOGD("Start Y: %d", startY);
LOGD("Canvas Hor: %d", canvasHorSize);
LOGD("Canvas Ver: %d", canvasVerSize);
LOGD("Draw Hor: %d", drawSizeHor);
LOGD("Draw Ver: %d", drawSizeVer);*/

//填充颜色给多余的地方
if(drawSizeHor < canvasHorSize || drawSizeVer < canvasVerSize){
    FPDFBitmap_FillRect( pdfBitmap, 0, 0, canvasHorSize, canvasVerSize,
                        0x848484FF); //Gray
}

int baseHorSize = (canvasHorSize < drawSizeHor)? canvasHorSize :
(int)drawSizeHor;
int baseVerSize = (canvasVerSize < drawSizeVer)? canvasVerSize :
(int)drawSizeVer;
int baseX = (startX < 0)? 0 : (int)startX;
int baseY = (startY < 0)? 0 : (int)startY;
int flags = FPDF_REVERSE_BYTE_ORDER;

if(renderAnnot) {
    flags |= FPDF_ANNOT;
}

FPDFBitmap_FillRect( pdfBitmap, baseX, baseY, baseHorSize, baseVerSize,
                    0xFFFFFFFF); //white
//将实际数据填入pdfBitmap中
FPDF_RenderPageBitmap( pdfBitmap, page,
                        startX, startY,
                        (int)drawSizeHor, (int)drawSizeVer,
                        0, flags );
//将bitmap的颜色格式转换成RGB_565
if (info.format == ANDROID_BITMAP_FORMAT_RGB_565) {
    rgbBitmapTo565(tmp, sourceStride, addr, &info);
    free(tmp);
}

//解锁bitmap像素
AndroidBitmap_unlockPixels(env, bitmap);
}

```

## include下的fpdf文件

可以[查看Pdfium \(编译前的c++库\)](#)直接查看原始c++库源码，包括.h和.cpp

## Android.mk & Application.mk

- Android.mk
  - 常见的c++\_shared和jni主文件
  - 引入了pthread，做线程锁
  - 引入了[pdfium的原始库的编译后文件，以及配套在include下fpdf的头文件](#)
  - 引入了android下的graphics包，方便做bitmap处理

- 引入了**freetype2**的so库**libmodft2**，做字体渲染 [freetype-Github](#)
  - 引入了**libpng**的so库**libmodpng** [libpng-Github](#)
- Application.mk
  - 常见的abi编译版本设置和c++\_shared
  - 引入了**Exception**，方便jni层中抛出错误，但c++层无法抛出
  - 根据**ndk-build**的版本的的不同可能存在 mips/mips64无法打包编译的问题，毕竟这两个版本相对太久了

## AndroidPdfViewer

[GitHub源地址](#)

编译所需修改配置同上

基于[PdfiumAndroid](#)之上，对获取到的pdf的native层指针docPtr进行操作，并且将其渲染在bitmap上

## CacheManager

维护两个优先级队列，**activeCache**和**passiveCache**

**activeCache**主要是用来存储**PagePart**类（用于存储pdf当前页的bitmap以及页码等其他信息），并且只存储最新的获得到

**passiveCache**存储非最新获取到的**PagePart**类

## RenderingHandler

本身是一个Handler

- openPage
  - 可以直接看上面的[mainJNI Lib.cpp](#)的**nativeLoadPage**方法
- proceed
  - 通过handleMessage中接收**RenderingTask**类（存储要渲染的pdf信息，例如绘制质量，大小宽高等）
  - 接着通过**proceed**方法进行预处理

```
通过初始化传入的pdf信息，先将页码pageIndex转换成native层的指针private PagePart
proceed(RenderingTask renderingTask) throws PageRenderingException {
    // 获取pdfView中已经打开的pdf，并且获得此页在pdf中的索引
    PdfFile pdfFile = pdfView.pdfFile;
    // 通过初始化传入的pdf信息，先将页码pageIndex转换成native层的指针
    pdfFile.openPage(renderingTask.page);

    int w = Math.round(renderingTask.width);
    int h = Math.round(renderingTask.height);

    if (w == 0 || h == 0 || pdfFile.pageHasError(renderingTask.page)) {
        return null;
    }

    Bitmap render;
```

```

        try {
            // 根据选择的bitmap质量来进行bitmap的初始化
            render = Bitmap.createBitmap(w, h, renderingTask.bestQuality ?
            Bitmap.Config.ARGB_8888 : Bitmap.Config.RGB_565);
        } catch (IllegalArgumentException e) {
            Log.e(TAG, "Cannot create bitmap", e);
            return null;
        }

        // 根据矩阵和四舍五入计算出此页pdf的边界
        calculateBounds(w, h, renderingTask.bounds);

        // 通过native层对此页pdf的bitmap进行渲染绘制，详细查看上文nativeRenderPageBitmap
        pdfFile.renderPageBitmap(render, renderingTask.page, roundedRenderBounds,
        renderingTask.annotationRendering);

        return new PagePart(renderingTask.page, render,
            renderingTask.bounds, renderingTask.thumbnail,
            renderingTask.cacheOrder);
    }
}

```

- 最后再进行[下面的绘制操作，详细请看绘制过程](#)

## 绘制过程

1. 首先通过外部调用onBitmapRendered来获取最新的PagePart类
2. 然后就最新的PagePart类进行CacheManager的cachePart处理，也即是将现有的数据从activaCache复制到passiveCache中，再将新的PagePart存储到activaCache队列中
3. 调用invalidate进行刷新
4. 在onDraw中先进行位置居中复位
5. 然后计算PagePart中的bitmap整体通过Canvas.nDrawBitmap进行复制到指定区域并绘制

```

//PdfView.java
public void onBitmapRendered(PagePart part) {
    // 首帧处理
    if (state == State.LOADED) {
        state = State.SHOWN;
        callbacks.callOnRender(pdfFile.getPagesCount());
    }

    if (part.isThumbnail()) {
        cacheManager.cacheThumbnail(part);
    } else {
        //通过CacheManagr对PagePart进行缓存
        cacheManager.cachePart(part);
    }
    // 调用刷新绘制
    redraw();
}

```

...

```

@Override
protected void onDraw(Canvas canvas) {

    ...

    // 位置居中复位
    float currentXOffset = this.currentXOffset;
    float currentYOffset = this.currentYOffset;
    canvas.translate(currentXOffset, currentYOffset);

    ...

    // pdf进行绘制
    for (PagePart part : cacheManager.getPageParts()) {
        drawPart(canvas, part);
        ...
    }

    ...

    // 回归之前换页前的位置
    canvas.translate(-currentXOffset, -currentYOffset);
}
...

private void drawPart(Canvas canvas, PagePart part) {
    // 计算当前此页pdf的大小区域
    RectF pageRelativeBounds = part.getPageRelativeBounds();
    Bitmap renderedBitmap = part.getRenderedBitmap();

    if (renderedBitmap.isRecycled()) {
        return;
    }

    // 所有的pdf均绘制在同一张bitmap中
    // 根据当前页pdf的页码计算出应在bitmap中的位置，然后进行位移
    float localTranslationX = 0;
    float localTranslationY = 0;
    SizeF size = pdfFile.getPageSize(part.getPage());
    if (swipeVertical) {
        localTranslationY = pdfFile.getPageOffset(part.getPage(), zoom);
        float maxWidth = pdfFile.getMaxPageWidth();
        localTranslationX = toCurrentScale(maxWidth - size.getWidth()) / 2;
    } else {
        localTranslationX = pdfFile.getPageOffset(part.getPage(), zoom);
        float maxHeight = pdfFile.getMaxPageHeight();
        localTranslationY = toCurrentScale(maxHeight - size.getHeight()) / 2;
    }
    canvas.translate(localTranslationX, localTranslationY);
    float offsetX = toCurrentScale(pageRelativeBounds.left * size.getWidth());
    float offsetY = toCurrentScale(pageRelativeBounds.top * size.getHeight());
    float width = toCurrentScale(pageRelativeBounds.width() * size.getWidth());

```



```

float height = toCurrentScale(pageRelativeBounds.height() * size.getHeight());

// pdf中的bitmap实际大小
Rect srcRect = new Rect(0, 0, renderedBitmap.getWidth(),
    renderedBitmap.getHeight());

RectF dstRect = new RectF((int) offsetX, (int) offsetY,
    (int) (offsetX + width),
    (int) (offsetY + height));

...

//将pdf的bitmap绘制在dstRect的指定位置中, 大小为srcRect
canvas.drawBitmap(renderedBitmap, srcRect, dstRect, paint);

if (Constants.DEBUG_MODE) {
    debugPaint.setColor(part.getPage() % 2 == 0 ? Color.RED : Color.BLUE);
    canvas.drawRect(dstRect, debugPaint);
}

// Restore the canvas position
canvas.translate(-localTranslationX, -localTranslationY);
}

```

## PdfRenderer

[PdfRenderer](#) | [API reference](#) | [Android Developers](#)

本质上也是通过`ParcelFileDescriptor`生成对应pdf文件的文件索引fd, 然后再把fd传的`PdfRenderer`中, 最后再创建对应的bitmap数组来存放PdfRenderer所读取到每一页的内容

自带线程安全, 在`android>35`之后会有一个`FormEditRecord`进行pdf修改记录

```

//通过ParcelFileDescriptor生成对应pdf文件的文件索引fd
ParcelFileDescriptor fd = null
try {
    fd = ParcelFileDescriptor.open(file, ParcelFileDescriptor.MODE_READ_ONLY);
} catch (FileNotFoundException ex) {

}

//fd传的PdfRenderer
PdfRenderer renderer = null;
try {
    renderer = new PdfRenderer(fd);
} catch (IOException ex) {

}

//创建对应的bitmap数组来存放PdfRenderer所读取到每一页的内容

```

```
int width = 1080; // 单页pdf的宽
int height = 1920; // 单页pdf的高
int pdfPageCount = renderer.getPageCount(); // 获取pdf的总页码
List<Bitmap> bitmapList = new ArrayList<Bitmap>(pdfPageCount); // 存放所用的bitmap数组
int pageCount = 0
while (pageCount < pdfPageCount) {
    // 打开当前pdf的第pageCount页
    PdfRenderer.Page page = renderer.openPage(pageCount);
    Bitmap pageBitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888);
    // 将此页pdf渲染到bitmap中
    page.render(pageBitmap, null, null, PdfRenderer.Page.RENDER_MODE_FOR_DISPLAY);
    page.close();
    pageCount++;
}
```

## Pdfium (编译前的c++库)

---

[pdfium - Git at Google](#)

[GitHub - documentcloud/pdfium: A mirror of PDFium \(primary source https://pdfium.googlesource.com/pdfium/\).](#)

## 具体到项目中实现

---

pdfmodule